Forensics Text Completion with Transformer-based Models: Fine tuning and Retrieval-Augmented Approach

Pravesh Srivastava[1] and Ravi Saharan[2]

Dept. of CSE, Central University of Rajasthan, Ajmer, India.
papsri.2015@gmail.com, ravisaharan@curaj.ac.in

Abstract. In today digital era forensic report files sometimes contain missing, unstructured or corrupted information. Handling such files is a difficult task for security analysts, especially when they need to respond quickly. Creating a framework that can automatically complete missing information and correct corrupted documents effectively can help investigators by reduces time, efforts, minimize errors and helps in understanding vulnerability effectively. In our paper, we proposed method to complete missing texts in the field of digital forensics. For this we tested two types of transformer based model: decoder-only models like GPT-2 and encoder-decoder models like T5 and BART. In this whole process, we first collected the data from multiple sources and build the custom dataset. Then, FAISS indexing was performed for retrieval, this is input to the generator model and then evaluated using metrics such as BLEU, ROUGE, PPL, and BERTScore. We compared the baseline and RAG variants of generative models, where the model has to write short summaries of vulnerability and completing the missing text. In our training and testing phase we find out that decoder-only models like GPT-2 gives better result with score of 91% when compared to encoder-decoder model.

Keywords: RAG, Transformer, GPT-2, T5, BART, FAISS.

1 Introduction

In digital forensic, while investigating the case, security analysts often face situations where they need to deal with huge amount of unstructured data (specifically textual data). In forensic domain it could be CVE report files, vendor advisories, incident tickets and more. Documents like this plays very important role in investigating the digital crimes, as they can provide crucial information regarding the events that happens in particular case. However, many of these records are very lengthy, incomplete and sometimes corrupted. This makes the job difficult as it takes huge amount of time dealing with unstructured and corrupted files. Automating these tasks can reduces the workload of analysts and improves the efficiency of the whole investigation. This automation includes filling the missing details, summarizing unstructured text, handling corrupted files effectively, making the overall process reliable and efficient. Transformer based models are already effective and reliable in wide array of NLP tasks and therefore they can also be used for forensic text completion. However, when these models generate text without any external support, they produce wrong outputs and

sometimes hallucinate over incomplete text. Solution to this problem is RAG (Retrieval Augmented Generation), as it improve model by combining them with external support, rather than generating answers only from models memory. RAG allows model to first retrieve relevant information from trusted sources (this includes CVE/NVD database and the MITRE ATT&CK dataset), then model generates content based on retrieved and memory it has. In simple terms, RAG provides model reliability and more accurate generations with real and up-to-date forensic knowledge. This makes the final output trustworthy, especially when working with corrupted and incomplete forensics data.

2 Related Work

Sufi et al. (2024) submitted research paper on Generation Pretrained Transformer (GPT) in Research for data augmentation task. Their work shows that GPT can helps in generating and analyzing with research data, this is useful for researchers who want to use GPT to for their work, make their research more efficient, and solve problems in design, they also mentioned that GPT does have some issues, like biases, and inaccuracies (called hallucinations) which can reduce its usefulness for research [1].

Hwang et al. (2024) introduced DSLR (Document Refinement with Sentence-Level Re-ranking and Reconstruction) a new unsupervised framework that improves RAG systems used with LLM models. RAG systems combine external knowledge with model generation but sometimes struggle with missing information. DSLR fixes this by breaking documents into sentences, ranking them, and rebuilding them into clear, useful passages. Their work shows DSLR performs better than traditional methods without extra training [2].

Gao et al. (2024) reviews Retrieval-Augmented Generation (RAG), provided a method that improves LLM models by using external knowledge sources. LLMs are powerful but face problems like hallucinations, outdated information. RAG fix these issues by combining external knowledge source. They also explained the key parts of RAG, such as retrievers and retrieval fusion methods, and compares their strengths and weaknesses [3].
Costin et al. (2023) studies how well ChatGPT can predict Common Vulnerabilities and Exposures (CVE) data from plain text descriptions without extra training, using a method called zero-shot learning. The researchers tested ChatGPT on 115,000 CVE entries from 2002 to 2023, collected from the Fast CVE database, their study shows ChatGPT's zero-shot performance in prediction moderate but lacks the accuracy. The authors conclude that Large Language Models cannot replace human security experts for vulnerability analysis at least for now [4].

He et al. (2021) introduces DeBERTa (Decoding-enhanced BERT with disentangled attention), a better version of BERT and RoBERTa. It uses two new ideas disentangled attention, where each word represented by two vectors for meaning and position, and an enhanced mask decoder that helps predicting the missing words more accurately, and uses a training method called Virtual adversarial training which makes model reliable [5].

Mittal et al. (2021) proposed a survey for the identification of cybersecurity attacks by applying NLP methods. They performed experiment on existing approaches and concluded that the normal models are not effective but deep learning and transformer achieve better performance in prediction of cyberattacks [6].
Boucher et al. (2020) performed the work of automatic extraction of vulnerability descriptions from security advisories. Their provided approach improves the retrieval of the data from documents and the efficiency in automating and reducing the manual work involved in cybersecurity [7].
Lewis et al. (2020) proposed Retrieval-Augmented Generation (RAG), a model that combines two types of memory 'parametric' (stored knowledge in the model) and 'non-parametric' (external data like Wikipedia). The model performed better than regular models on open-domain question answering tasks, While RAG reduces false or made-up answers, it still depends on the accuracy of external data sources [8].
Zhang et al. (2020) introduced BERTScore, for evaluating text generation by comparing how similar words are in meaning, not just by exact matches. They tested on various translation and captioning systems, BERTSCORE aligns better with human perspective and handles tricky examples more reliably than other metrics. BERTSCORE is simple, flexible, and effective for evaluating text quality in NLP tasks [9].
Raffel et al. (2020) introduced T5, a text-to-text transformer model. Instead of treating each task differently, it converts every problem into summarization, question answering, and takes a text input and text output. Paper concludes, using a large new dataset called the Colossal Clean Crawled Corpus, T5 achieved state-of-the-art results on many benchmarks [10].
Radford et al. (2019) proposed GPT-2, a large language model (LLM) trained on a massive dataset called WebText. GPT-2 learns to perform tasks like question answering, summarization, and translation without specific training—this is called zero-shot learning. GPT-2's performance improves as its size increases, showing that larger models learn better general skills. In conclusion, the study shows that large-scale unsupervised training can teach models to perform complex tasks efficiently [11].

Devlin et al. (2019) presented BERT (Bidirectional encoder representations from transformers), language model that reads text in both directions at the same time. It is first trained on lots of text without labels, and then can be easily fine-tuned for specific tasks like question answering with just one extra layer. BERT achieves top results on many benchmarks, improving performance on tasks like GLUE, MultiNLI, and SQuAD [12].
Lewis et al. (2019) created BART (Bidirectional and Auto-Regressive Transformer) as a sequence-to-sequence model trained for reducing the noise. They showed that how effective BART was for tasks like summarization and translation and other generative tasks. BART beats other models like RoBERTa on many tasks and sets new records for text generation. It's reliable and volatile and can easily be fine tuned for various NLP tasks [13].
Reimers et al. (2019) introduced Sentence-BERT, SBERT uses siamese and triplet networks to create sentence embeddings that can be compared with cosine similarity. SBERT works well on semantic similarity and transfer learning. It is faster, more reliable than other sentence embeddings methods [14].

RoBERTa was introduced by Liu et al. (2019) as a better BERT in consideration of improved training processes like bigger batches and longer training times. They indicated the advantage of RoBERTa over BERT and the need for careful training decisions in transformer models [15].

3 Methodology

For this work data collection is one of the major issues and converting this data into model specific format is another complicated task. Since each model takes input in a different format. The first task is data collection and converting this into model specific format then, we'll train our generator models with and without RAG. In the end of this discussion, we'll compare all the models and find out which model performs the best.

3.1 Proposed work

The basic pipeline where we first preprocess the datasets and create corpus for RAG will input to our generator models and their responses are evaluated for the metrics like (PPL, BLEU, BERTScore, ROUGE).

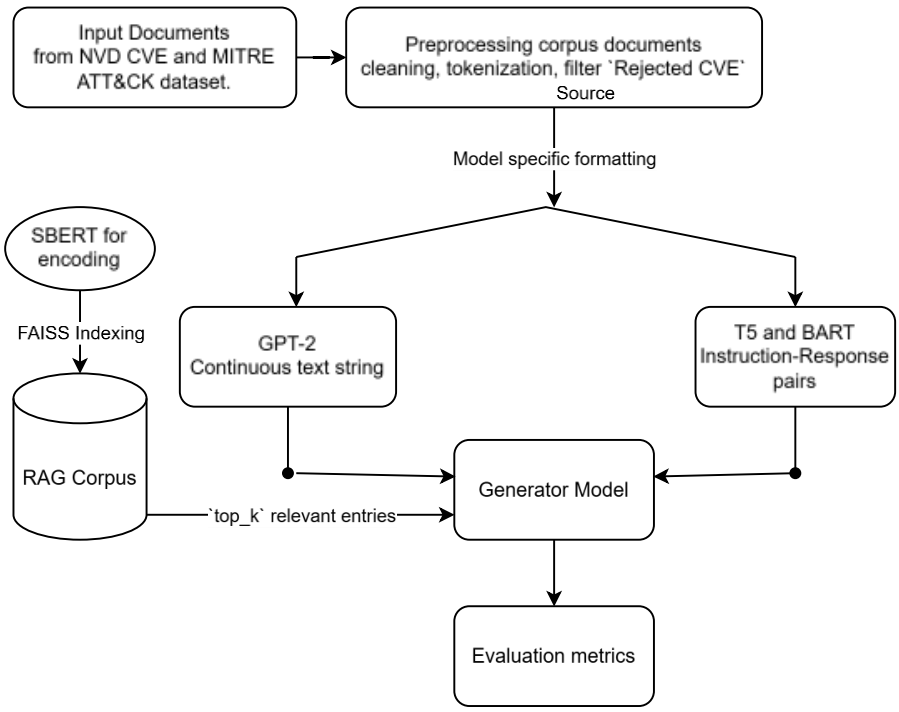

Fig. 1. Proposed architecture

3.2 Dataset Collection

We used publicly available cybersecurity datasets. 1. *NVD CVE* dataset obtained in (.jsonl) format, which contains structured vulnerability information such as CVE-ID, severity score, and description of the issue. 2. *MITRE ATT&CK* dataset also obtained in (.jsonl) format from GitHub, providing details of adversarial techniques, attack patterns, and malware descriptions.

From the raw data files (.jsonl) format, we've extracted the CVE-ID, severity score and description of the vulnerability. This extracted information then later converted into model specific format for training. In our case decoder only models like GPT-2 expects the input single continuous text string whereas the encoder-decoder style models like T5 and BART expect each sample in instruction-response pair samples.

Later we've merged the NVD CVE (.jsonl) file and MITRE ATT&CK (.jsonl) to create the corpus also in raw (.jsonl) format used as retrieval knowledge base, by doing this we're ensuring that for each unseen datapoint we've similar looking datapoint which helps in retrieval generation when we'll generate *top_k* no. of response from the model during inference. During inference, when a query is given to the model, the retrieval module searches the corpus file and retrieves the *top_k* most relevant entries. These retrieved descriptions are then provided to the model as additional context, allowing it to generate more accurate and informed responses.

3.3 Dataset Preparation

Like we've section above, the exact same approach is used for the preparation of the data in model specific format for testing and training.

3.3.1 Data Extraction and sampling: From the raw (.jsonl) files, we extracted three key fields for every entry, CVE-ID, severity, textual description.

After extraction, we constructed a dataset of *10K* samples, out of these, *7K* samples were randomly drawn from the NVD CVE dataset and remaining *3K* samples were selected from the MITRE ATT&CK dataset.

The combined dataset was then divided into three splits, *n_train* which contains 8K samples used for training, *n_val* used for validation contains *1K* samples *n_test* used for testing also contains *1K* samples. This random sampling and splitting ensured a balanced and representative dataset for model training and evaluation.

3.3.2 Formatting for model training: Different language models expect different types of input formats. To make the data model-ready, we designed separate formatting pipelines.

For GPT-2 each entry was converted into a single continuous text string containing the description (e.g., *malware description or CVE entry*).

For T5 and BART each sample was structured into an instruction–response pair. The instruction part used a set of prompt templates (e.g., *Summarize the vulnerability report*, *Expand the threat description*), and the response contained the corresponding vulnerability description.

This formatting step allowed the models to learn in a way that matched their native training styles: autoregressive text generation for GPT-2 and instruction-based completion for T5 and BART.

3.3.3 Corpus construction for RAG: In addition to preparing training data, we also created a *retrieval corpus* for enabling Retrieval-Augmented Generation (RAG). For this, all entries from both NVD CVE and MITRE ATT&CK datasets were merged into a single file called *cve_corpus.jsonl*. In a real-world scenario, the retrieval system would fetch supporting information directly from the web or an external knowledge base. However, to keep the setup simple and locally reproducible, we stored all records in this merged corpus file.

4 Experimental setup and Results

In our work, the "Baseline model" means a pretrained transformer based LLM (GPT-2, T5, or BART) that generates text from its parametric memory. The "RAG model" means the same LLM but with extra outside memory added using retrieval. The model takes the top-k retrieved passages and adds them to the input before generating text.

So, our comparison is between a transformer based models which uses its parametric memory and same model with retrieval added non-parametric memory along with its trained memory.

We fine-tune these models for the task of forensic text completion. Each model was trained and evaluated in two different settings:

1. without retrieval, where the model only generates based on its learned parameters,
2. with Retrieval-Augmented Generation (RAG), where the model also uses external knowledge from CVE and MITRE ATT&CK datasets during generation.

The aim was to compare how retrieval affects the quality of the generated outputs and to identify which model performs best in this domain.

All three models GPT-2, T5, and BART were fine-tuned on the training dataset *n_train* using the same setup. For the retrieval-based version, we implemented a simple RAG pipeline where the model retrieved relevant passages before generating the output. This was done using FAISS indexing, which helped in finding the most relevant text from the knowledge base. The baseline variant of each model is trained on without retrieval dataset, so that we can compare it with the RAG variant of the same model.
All the models are trained on google colab T4 GPU using the Hugging Face library and hyperparameter tuning is done on each model for finding best parameter values.

- GPT-2 is fine-tuned for 3 epoch values with a learning rate of 5e-5, the batch size of 2 is used for both training and testing, and at last the weight decay is set to 0.01.

- Similar to GPT-2 T5 also trained for 3 epoch values with the same learning rate and batch size. along with this it also uses gradient accumulation with step

size of 4 and mixed precision (fp16) in training phase. Checkpoints are saved after every iteration and loss is logged after 500 steps.

- Unlike GPT-2 and T5, BART on the other hand fine-tuned for 5 epoch values with same learning rate. its batch similar to GPT-2 and gradient accumulation similar to T5, and same mixed precision (fp16) used here for efficiency. Here also Checkpoints are saved after each iteration and loss is logged after 200 steps.

For evaluation of above models, metrics like PPL, BLEU, ROUGE, BERTScore are commonly used is text generation task.

For T5 with RAG, the model achieved a perplexity of 11.88. The BLEU score is 6.09%, and the ROUGE scores are (ROUGE-1 = 6.23%, ROUGE-2 = 8.65%, ROUGE-L = 11.7%). BERTScore are, Precision = 78.74%, Recall = 81.6%, and F1 score = 79.9%. Higher BERTScore values represents that T5 produced semantically related completions, it struggled to match surface-level word overlap.

For BART with RAG, the results decreased in terms of lexical overlap. The BLEU score is 2.97%, and ROUGE scores are (ROUGE-1 = 12.6%, ROUGE-2 = 3.2%, ROUGE-L = 10.1%). BERTScore values is better when compared to T5, with Precision = 82.71%, Recall = 81.3%, and F1 score = 81.6%, this shows BART gives better accuracy and contextual relevant details as compared to T5, but it did not perform well when compared to GPT-2.

The GPT-2 with RAG model outperformed both T5 and BART across all metrics. It achieved a very low perplexity of 1.84, showing that it was highly confident in generating coherent text. BLEU score on the other hand is higher when compared to other models, 33.03% along with the ROUGE score of (ROUGE-1 = 48.4%, ROUGE-2 = 37.9%, ROUGE-L = 44.67%). BERTScore confirms that with Precision = 87.5%, Recall = 96.05%, and F1 score = 91.05%. RAG variant of GPT-2's has stronger ability to generate semantic, lexical, and coherent forensic text completions.

Conclusion from our experiment, adding retrieval improves the quality of the generated text across the models. In our case, GPT-2 with RAG performed the best when compared to T5 and BART, it shows highest accuracy in producing meaningful completions.
GPT-2 performs better because decoder-only models are very good at open-ended text generation and make better use of the retrieved context. T5 and BART are encoder–decoder models, which usually work best for more structured tasks, not long free-form text completion like in our work.

Hence, we can safely conclude that decoder-only models (like GPT-2), when combined with retrieval, increases the performance of text completion task, when compared to encoder-decoder models (like T5 and BART).

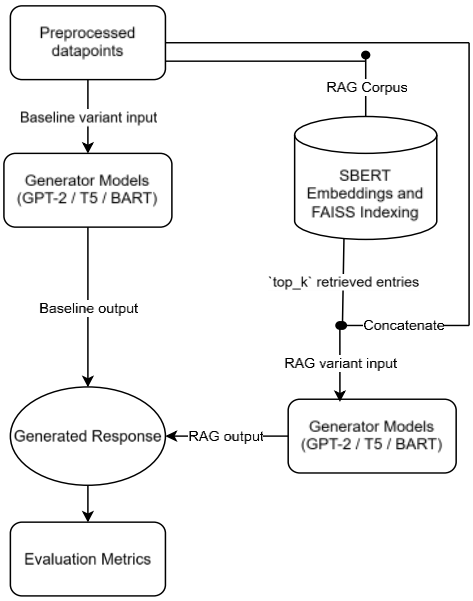

Fig. 2. Model training pipeline for both RAG and Baseline variants

Results

From Table 1. GPT-2 seems to work better because decoder-only transformer models are designed for open-ended text generation and gain more from retrieval.
RAG variant of GPT-2 outperformed when compared to other model and their RAG variants, it is able to achieve the highest BERTScore, and the lowest BLEU and PPL scores, which indicates its output is closest to the reference text while preserving the semantics meaning. The baseline variant of GPT-2 is able to achieve a better ROUGE-L value but its BLEU and BERTScore confirms that RAG helps GPT-2 in generating texts effectively. On the other hand, T5 and BART did not perform well as compared to GPT-2, this we can clearly see from their BLEU, PPL and BERTScore metrics. This makes them less suitable for this task. Finally, we can conclude that GPT-2 integrated with RAG is most robust and reliable model.

Table 1. Accuracy of RAG vs Baseline model

Model	**PPL**	**BLEU**	**ROUGE-L**		**BERTScore**	
				Precision	**Recall**	**F1**
GPT-2 (RAG)	1.84	33.03	44.67	87.5	96.05	91.0
GPT-2	2.18	28.5	45.7	86.60	82.46	84.9
T5 (RAG)	11.88	6.09	11.7	78.74	81.6	79.9
T5	20.12	5.58	5.46	67.7	81.23	73.8
BART (RAG)	8.09	2.97	10.1	82.71	81.3	81.6
BART	7.83	2.45	8.19	76.6	80.72	78.6

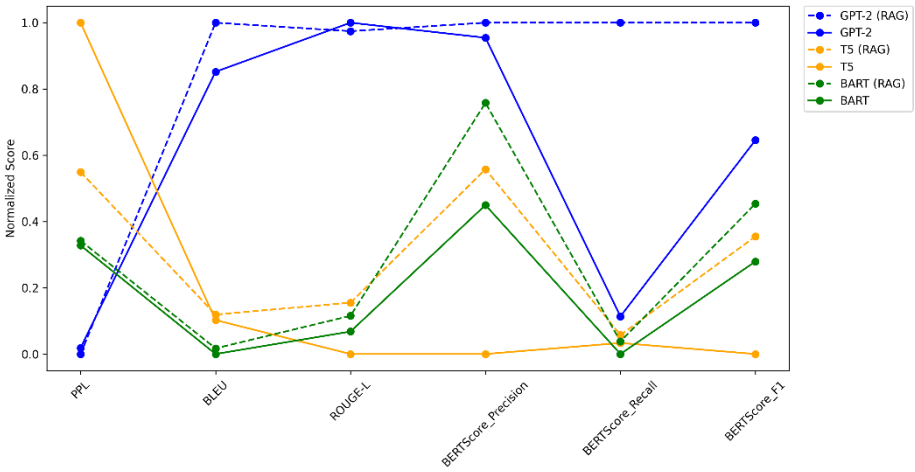

Fig. 3. Comparison Baseline vs RAG Models

Above Fig 3. represents comparison between baseline variant (dashed lines) and RAG variant (solid lines) of model, each color represents a different model family (as mentioned in the label of Fig. 3). The y-axis represents the normalized scores for making the comparison easier, because of normalized scores the model that performed the best get closer to the top of the graph. We can clearly see that among all models RAG variants perform better when compared to their baseline variants. As we concluded in the results section that RAG variant of GPT-2 performs the best as compared to all the other models, we can see the exact same result from the above figure (Fig. 3) as well. This shows adding Retrieval to baseline model improves their overall performance which leads the improved quality of generated completions.

5 Conclusion and Future work

In this work, we showed that fine tuning transformer based models specifically decoder-only models like GPT-2 performs better especially for text generation task. Our experiment shows that adding Retrieval with these model shows noticeable improvements in the quality of generated outputs. We combined SBERT embedding with FAISS for fast similarity check and then using these to generate the final output. Our pipeline provides simple, robust and effective approach. RAG variant of GPT-2 shows the best performance across all the evaluation metrics. This represents decoder-only models performs the better specifically for text generation task.

For future work, we suggest to add a better re-ranking step, so that system can choose the most relevant passages effectively, instead of simply relying on similarity score. Another is integrating a huge corpus domain specific knowledge base that helps in better generation. Finally, more structured decoding methods can also be explored. This would make the generated reports more consistent and easier to use in practical scenarios.

References

1. Sufi, F. “Generative Pre-Trained Transformer (GPT) in Research: A Systematic Review on Data Augmentation.” (2024).
2. Hwang, H., Kim, Y., and Park, J. “DSLR: Document Refinement with Sentence-Level Re-ranking and Reconstruction to Enhance Retrieval-Augmented Generation.” Proceedings of Knowledge NLP Workshop, (2024).
3. Gao, Y., Li, J., and Sun, M. “Retrieval-Augmented Generation for Natural Language Processing: A Survey.” (2024).
4. Costin, A., Francillon, A., and Garcia, S. “Evaluating Zero-Shot ChatGPT Performance on Predicting CVE Data from Vulnerability Descriptions.” (2023).
5. He, J., et al. "DeBERTa: Decoding-enhanced BERT with Disentangled Attention." (2021).
6. Mittal, S., et al. "Cybersecurity Threat Detection Using NLP: A Survey." (2021).
7. Boucher, C., et al. "Automated Extraction of Vulnerability Descriptions from Security Advisories." (2020).
8. Lewis, P., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP.” (2020).
9. Zhang, T., et al. “BERTScore: Evaluating Text Generation with BERT.” (2020).
10. Raffel, C., et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." (2020).
11. Radford, A., et al. “Language Models are Unsupervised Multitask Learners.” (2019).
12. Devlin, J., et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." (2019).
13. Lewis, M., et al. “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.” (2019).
14. Reimers, N. and Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” (2019).
15. Liu, Y., et al. "RoBERTa: A Robustly Optimized BERT Pretraining Approach." (2019).